

Lecture 17: Callback Hell, Promises & Async/Await

Core Principles:

The two guiding principles behind writing better code:

- DRY (Do Not Repeat Yourself)
- Code Readability

Understanding Callback Hell :

Example (food delivery app): a food delivery app that requires tasks to execute in a sequence:

1. Place order → Wait → Get order ID
2. Prepare food → Wait → Food ready
3. Pickup order → Wait → Order picked up
4. Deliver order → Wait → Food delivered

Each step is asynchronous (it uses `setTimeout` to simulate the wait), and each step must fully finish before the next one starts.

Data Object:

To pass real order data through each step, an `orderDetail` object is created:

```
const orderDetail = {  
  orderId: 12314,  
  food: ["Pizza", "Burger"],  
  price: 900,  
  paymentStatus: false,  
  tokenAssign: false,  
  restaurantLocation: "New Delhi",  
  customerLocation: "Dwarka",  
  pickingOrder: false  
};
```

Since each order is unique, `orderDetail` is sent along with every function call:

Writing Each Step:

Step 1: Place order → Wait → Get order ID

```
function placeOrder(orderDetail, Callback){  
  console.log(`Payment of ${orderDetail.price} is in Progress`);  
  
  setTimeout(()=>{  
    console.log("Payment is completed");  
    orderDetail.paymentStatus = true;  
    Callback(orderDetail);  
  },3000)  
}
```

Step 2: Prepare food → Wait → Food ready

```
function prepareOrder(orderDetail, Callback){
  console.log(`Restaurant received the order and is preparing ${orderDetail.food}`);
  setTimeout(()=>{
    console.log("Your Order is prepared now");
    orderDetail.tokenAssign = 123;
    Callback(orderDetail);
  },3000)
}
```

Step 3: Pickup order → Wait → Order picked up

```
function pickupOrder(orderDetail, Callback){
  console.log(`Delivery boy is on the way to pick up the order from ${orderDetail.restaurantLocation} restaurant`);
  setTimeout(()=>{
    console.log("Delivery boy reached the restaurant");
    orderDetail.pickingOrder = true;
    Callback(orderDetail);
  },3000)
}
```

Step 4: Deliver order → Wait → Food delivered

```
function deliverOrder(orderDetail){
  console.log(`Delivery boy picked the order from restaurant and delivering it to ${orderDetail.customerLocation}`);
  setTimeout(()=>{
    console.log("Order Delivered");
  },1000)
}
```

Full Nested Chain:

```
placeOrder(orderDetail, () => {
  prepareOrder(orderDetail, () => {
    pickupOrder(orderDetail, () => {
      deliverOrder(orderDetail);
    });
  });
});
```

As orderDetail gets updated at each step, the same updated object flows through: prepareOrder → pickupOrder → deliverOrder.

This deeply nested, callback-inside-callback-inside-callback structure is called Callback Hell or Pyramid of Doom. Complexity increases further as more steps are added.

Inversion of Control:

In the real world, different teams handle different steps: one team handles prepareOrder, another handles pickupOrder, and so on. Each step depends on the previous one calling it correctly. This dependency of one function's execution on another function is called Inversion of Control.

Promises:

Because JavaScript is single-threaded, asynchronous handling is mandatory to prevent the website from freezing. Promises solve Inversion of Control by acting as a guarantee: "Don't wait for me synchronously; I will deliver the data when ready".

1. Creation & States: A Promise is created using new Promise((resolve, reject)). Calling resolve() moves the state to fulfilled, while reject() moves it to rejected.

2. Consuming Promises:

- .then() runs when the promise successfully fulfills.
- .catch() catches any errors or rejections.
- .finally() always executes at the end, regardless of success or failure (commonly used for resource cleanup, like closing database connections).

3. Chaining: To chain .then() methods, writing return inside the block is strictly compulsory so the next step receives the promise.

4. API Edge Case: The fetch() API does not automatically treat HTTP-level failures (like a 404) as rejected promises. Developers must manually check if (!response.ok) and throw a custom error.

```
fetch("https://api.github.com/users?page=20")
  .then((response) => {
    if (!response.ok)
      throw new Error("Unable to fetch data");
    return response.json();
  })
  .then((data) => {
    console.log(data);
  })
  .catch((error) => {
    console.log(error)
  })
  .finally(() => {
    console.log("I am the final step")
  });
```

Solving Callback Hell with Promises:

```
function placeOrder(orderDetail) {
  console.log(`Payment of ${orderDetail.price} is in progress`);

  return new Promise((resolve, reject) => {
    setTimeout(() => {
      console.log("Payment is completed");
      orderDetail.paymentStatus = true;
      resolve(orderDetail);
    }, 3000);
  });
}

function prepareOrder(orderDetail) {
  // Update orderDetail.tokenAssign, then resolve(orderDetail)
}

function pickupOrder(orderDetail) {
  // Update orderDetail.pickingOrder, then resolve(orderDetail)
}

function deliverOrder(orderDetail) {
  // Log delivery status and resolve(orderDetail)
}
```

```

placeOrder(orderDetail)
.then((orderDetail) => {
    return prepareOrder(orderDetail);
})
.then((orderDetail) => {
    return pickupOrder(orderDetail);
})
.then((orderDetail) => {
    return deliverOrder(orderDetail);
})
.then((orderDetail) => {
    console.log("Final Order Details:", orderDetail);
});

```

placeOrder updates and returns orderDetail. Once that value changes, the same orderDetail flows through prepareOrder → pickupOrder → deliverOrder.

Async/Await & Error Handling:

Async/Await:

The async/await syntax is the simplest and most readable way to work with Promises, allowing asynchronous code to look exactly like synchronous code.

```

async function order() {
    const p1 = await placeOrder(orderDetail);
    const p2 = await prepareOrder(orderDetail);
    const p3 = await pickupOrder(orderDetail);
    const p4 = await deliverOrder(orderDetail);
    console.log(p4);
}

```

Error Handling with try/catch:

Since await does not have a built-in .catch() method, the logic must be wrapped in a try/catch block to handle potential network failures or promise rejections safely.

```

async function github() {
    try {
        const response = await fetch("https://api.github.com/users?per_page=20");
        if (!response.ok) {
            throw new Error("Unable to fetch data");
        }
        const data = await response.json();
        console.log(data);
    } catch (error) {
        console.log(error);
    }
}

```